

Q_Learning_Epsilon_Greedy

September 16, 2026

1 Lab 1: Q-Learning with an Epsilon-Greedy Policy

Aim

- Implement a Q-learning agent in a Gymnasium environment.
- Study how the number of training episodes affects cumulative reward.
- Compare heavy early exploration against later exploitation using a decaying epsilon-greedy policy.

Environment

- Taxi-v4 is a discrete, non-continuous grid world from Gymnasium's toy text set.
- A taxi picks up a passenger from one of four locations and drops them off at another.
- The state is a single index of taxi row, taxi column, passenger location, and destination, 500 states total.
- There are 6 actions: move south, north, east, west, pick up, drop off.
- Reward is -1 per step, -10 for an illegal pickup or drop-off, +20 for a correct drop-off.
- Transitions are deterministic, so a failed test episode means the policy has not converged, not bad luck.

```
[1]: import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import gymnasium as gym
```

Setting up the environment

- gym.make creates the Taxi-v4 environment.
- observation_space.n gives the state count, 500.
- action_space.n gives the action count, 6.
- Both spaces are discrete, so a plain Q-table of shape (states, actions) works, no function approximation needed.

```
[2]: env = gym.make("Taxi-v4")

number_of_states = env.observation_space.n
number_of_actions = env.action_space.n

print("Number of states:", number_of_states)
```

```
print("Number of actions:", number_of_actions)
```

Number of states: 500

Number of actions: 6

Hyperparameters and the epsilon schedule

- discount_factor is 0.99, higher than usual, since a good Taxi episode runs 10 to 15 steps and the +20 reward has to survive that much discounting.
- Each state-action pair uses its own learning rate, $\alpha = 1$ divided by its visit count, the sample-average update rule.
- A shrinking per-pair learning rate keeps the Q-table stable across 3000 state-action pairs instead of one fixed alpha overwriting rare estimates too hard.
- Epsilon starts at 1.0, so early episodes are pure exploration.
- Epsilon decays toward a floor of 0.01 and never reaches zero.
- The decay rate is fixed and identical across all five episode budgets.
- A 500-episode run ends while epsilon is still high, a 10000-episode run decays close to the floor, and that gap is the whole point of the comparison.

```
[3]: episode_configs = [500, 1000, 2000, 5000, 10000]

max_steps_per_episode = 200
discount_factor = 0.99
epsilon_start = 1.0
epsilon_min = 0.01
epsilon_decay = 0.0008

def train_agent(num_episodes, seed):
    random.seed(seed)
    np.random.seed(seed)

    local_env = gym.make("Taxi-v4")
    local_env.action_space.seed(seed)

    q_table = np.zeros((local_env.observation_space.n, local_env.action_space.
↪n))
    visit_counts = np.zeros_like(q_table)

    episode_rewards = []
    epsilon = epsilon_start

    for episode in range(num_episodes):
        state, info = local_env.reset(seed=seed + episode)
        total_reward = 0

        for step in range(max_steps_per_episode):
            if random.uniform(0, 1) < epsilon:
                action = local_env.action_space.sample()
```

```

        else:
            action = np.argmax(q_table[state])

            new_state, reward, terminated, truncated, info = local_env.
↪step(action)
            done = terminated or truncated

            visit_counts[state, action] += 1
            alpha = 1.0 / visit_counts[state, action]

            best_future_q = 0 if terminated else np.max(q_table[new_state])
            target = reward + discount_factor * best_future_q
            q_table[state, action] += alpha * (target - q_table[state, action])

            state = new_state
            total_reward += reward

            if done:
                break

            epsilon = epsilon_min + (epsilon_start - epsilon_min) * np.
↪exp(-epsilon_decay * episode)
            episode_rewards.append(total_reward)

        local_env.close()
        return q_table, episode_rewards

def test_agent(q_table, test_episodes=100, seed=2024):
    local_env = gym.make("Taxi-v4")
    successes = 0
    total_rewards = []

    for episode in range(test_episodes):
        state, info = local_env.reset(seed=seed + episode)
        done = False
        terminated = False
        episode_reward = 0

        while not done:
            action = np.argmax(q_table[state])
            state, reward, terminated, truncated, info = local_env.step(action)
            done = terminated or truncated
            episode_reward += reward

        if terminated:
            successes += 1

```

```

        total_rewards.append(episode_reward)

    local_env.close()
    return successes, (successes / test_episodes) * 100, np.mean(total_rewards)

```

Training procedure

- Each of the five episode budgets trains a fresh Q-table from zero.
- Every episode's total reward is recorded.
- Each configuration trains with 5 seeds, since start conditions are randomised every episode.
- Test success rate and mean test reward are averaged across those 5 seeds.
- The cumulative-reward and moving-average plots use one representative seed, 42, so five overlapping runs don't clutter the figure.
- Testing runs the greedy policy, epsilon = 0, for 100 episodes, and a success means terminated = True, an actual completed drop-off, not a timeout.

```

[4]: results = {}
training_seeds = [7, 42, 99, 142, 242]
representative_seed = 42

for num_episodes in episode_configs:
    representative_q_table = None
    representative_rewards = None
    success_rates_this_config = []
    mean_test_rewards_this_config = []

    for seed in training_seeds:
        q_table, episode_rewards = train_agent(num_episodes, seed=seed)
        _, success_rate, mean_test_reward = test_agent(q_table,
        ↪test_episodes=100, seed=2024)
        success_rates_this_config.append(success_rate)
        mean_test_rewards_this_config.append(mean_test_reward)

    if seed == representative_seed:
        representative_q_table = q_table
        representative_rewards = episode_rewards

    results[num_episodes] = {
        "q_table": representative_q_table,
        "episode_rewards": representative_rewards,
        "cumulative_reward": np.cumsum(representative_rewards),
        "success_rate": np.mean(success_rates_this_config),
        "mean_test_reward": np.mean(mean_test_rewards_this_config),
    }
    print(
        f"Episodes: {num_episodes:>6} | Final cumulative reward (seed 42): ↪
    ↪{results[num_episodes]['cumulative_reward'][-1]:>8.0f} "

```

```

        f"| Mean test success rate (5 seeds):␣
↪{results[num_episodes]['success_rate']:>5.1f}% "
        f"| Mean test episode reward:␣
↪{results[num_episodes]['mean_test_reward']:>7.2f}"
    )

```

Episodes: 500 | Final cumulative reward (seed 42): -272153 | Mean test success rate (5 seeds): 11.4% | Mean test episode reward: -175.82

Episodes: 1000 | Final cumulative reward (seed 42): -394144 | Mean test success rate (5 seeds): 16.6% | Mean test episode reward: -164.86

Episodes: 2000 | Final cumulative reward (seed 42): -504812 | Mean test success rate (5 seeds): 21.2% | Mean test episode reward: -155.12

Episodes: 5000 | Final cumulative reward (seed 42): -610990 | Mean test success rate (5 seeds): 32.6% | Mean test episode reward: -131.26

Episodes: 10000 | Final cumulative reward (seed 42): -666255 | Mean test success rate (5 seeds): 45.2% | Mean test episode reward: -104.97

Summary table

- Final cumulative reward is the seed-42 run's running total, and it is not comparable across configurations, a longer run just sums more terms.
- Average reward over the last 100 training episodes is comparable across configurations and is the better indicator of policy quality.
- Test success rate and mean test reward are both averaged over 5 seeds and measured under a purely greedy policy.

```

[5]: summary_rows = []
for num_episodes in episode_configs:
    r = results[num_episodes]
    summary_rows.append({
        "Training episodes": num_episodes,
        "Final cumulative reward (seed 42)": int(r["cumulative_reward"][-1]),
        "Avg reward (last 100 training eps)": np.mean(r["episode_rewards"][-100:
↪]),
        "Test success rate % (mean of 5 seeds)": r["success_rate"],
        "Mean test episode reward (mean of 5 seeds)": r["mean_test_reward"],
    })

summary_df = pd.DataFrame(summary_rows)
summary_df

```

```

[5]:   Training episodes  Final cumulative reward (seed 42)  \
0                500                -272153
1                1000                -394144
2                2000                -504812
3                5000                -610990

```

	4	10000	-666255
		Avg reward (last 100 training eps)	Test success rate % (mean of 5 seeds) \
0		-378.50	11.4
1		-183.79	16.6
2		-84.43	21.2
3		-22.84	32.6
4		-4.93	45.2
		Mean test episode reward (mean of 5 seeds)	
0		-175.824	
1		-164.860	
2		-155.122	
3		-131.262	
4		-104.968	

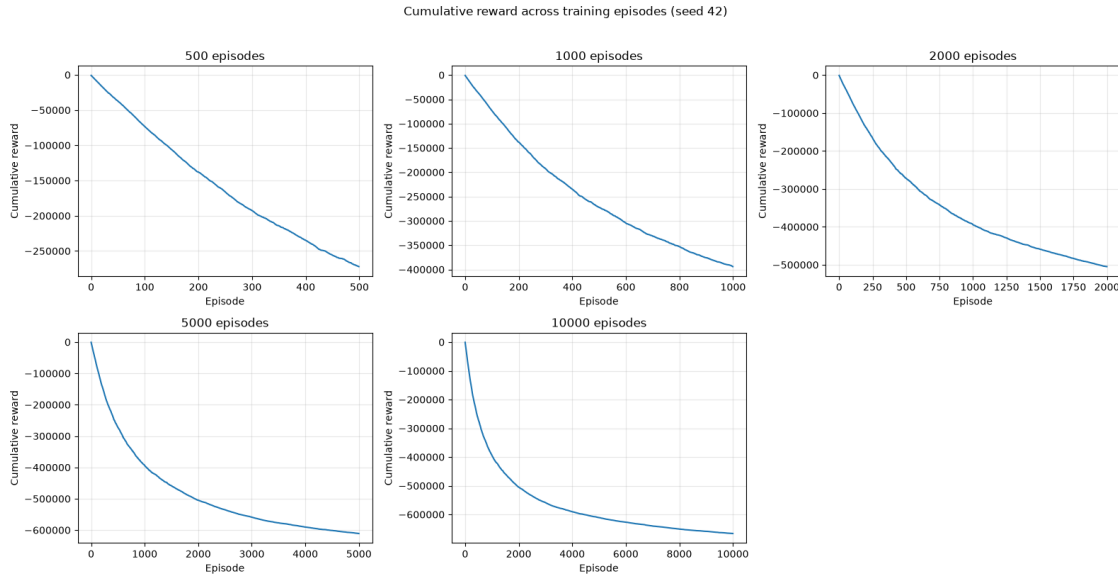
Cumulative reward vs training episodes

- Cumulative reward is the running sum of episode rewards.
- Taxi's -1 and -10 penalties mean this sum trends downward for most of training, it does not automatically rise.
- The slope is what matters here, it should flatten as the policy improves, even while the total stays negative.

```
[6]: fig, axes = plt.subplots(2, 3, figsize=(16, 8))
axes = axes.flatten()

for ax, num_episodes in zip(axes, episode_configs):
    r = results[num_episodes]
    ax.plot(r["cumulative_reward"], color="tab:blue")
    ax.set_title(f"{num_episodes} episodes")
    ax.set_xlabel("Episode")
    ax.set_ylabel("Cumulative reward")
    ax.grid(True, alpha=0.3)

axes[-1].axis("off")
fig.suptitle("Cumulative reward across training episodes (seed 42)", y=1.02)
fig.tight_layout()
plt.show()
```



Average reward over the most recent 100 episodes

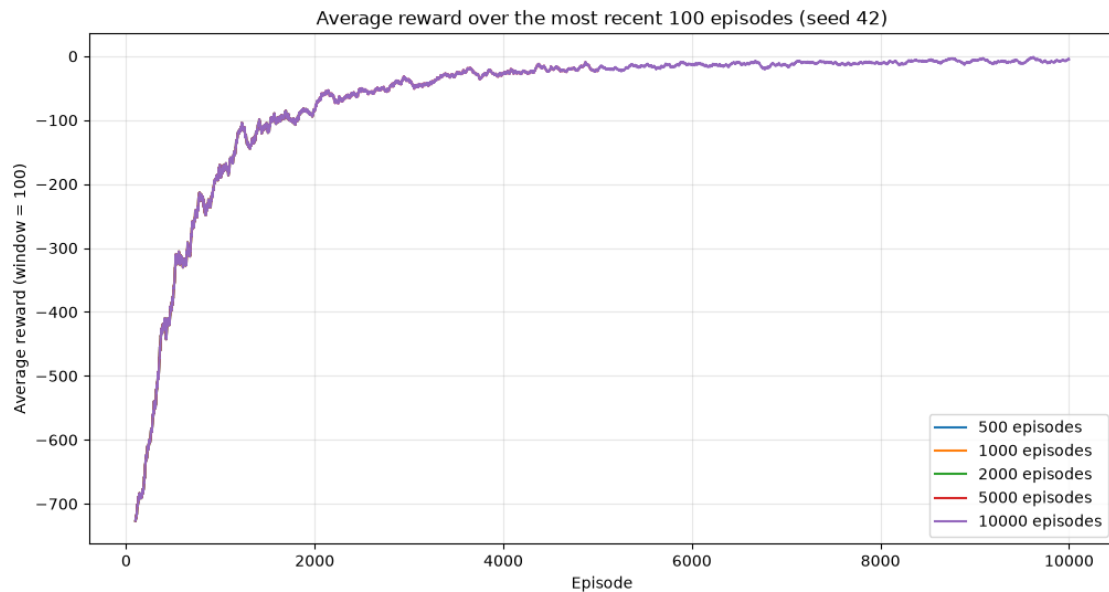
- Cumulative reward mixes early flailing with later competence, so it can't show current performance.
- A 100-episode rolling average shows that instead, it rises as the agent improves.
- For Taxi, the average climbs from deeply negative toward zero, and may not fully cross into positive territory at these episode counts.

```
[7]: window_size = 100

plt.figure(figsize=(12, 6))
for num_episodes in episode_configs:
    rewards = results[num_episodes]["episode_rewards"]
    if len(rewards) >= window_size:
        moving_avg = np.convolve(rewards, np.ones(window_size) / window_size,
        mode="valid")
        x = np.arange(len(moving_avg)) + window_size - 1
    else:
        moving_avg = np.array(rewards)
        x = np.arange(len(moving_avg))
    plt.plot(x, moving_avg, label=f"{num_episodes} episodes")

plt.title(f"Average reward over the most recent {window_size} episodes (seed_
42)")
plt.xlabel("Episode")
plt.ylabel("Average reward (window = 100)")
plt.legend()
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

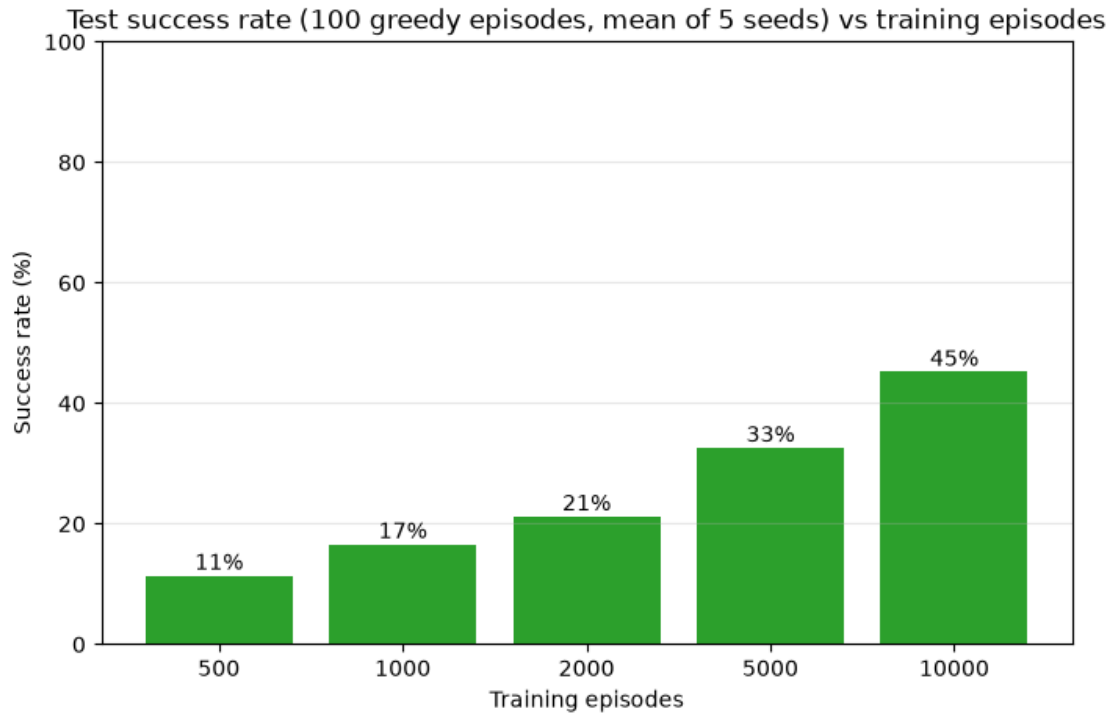


Test success rate after training

- Every trained Q-table is tested for 100 greedy episodes, and the bar height is the mean across 5 seeds.
- Taxi is deterministic, so a failed episode means the policy is incomplete, not unlucky.
- 500 states is a lot to cover, so a steady rise across configurations is expected rather than an early plateau.

```
[8]: success_rates = [results[n]["success_rate"] for n in episode_configs]

plt.figure(figsize=(8, 5))
bars = plt.bar([str(n) for n in episode_configs], success_rates, color="tab:
    ↪green")
for bar, rate in zip(bars, success_rates):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1, f"{rate:.
    ↪0f}%", ha="center")
plt.title("Test success rate (100 greedy episodes, mean of 5 seeds) vs training_
    ↪episodes")
plt.xlabel("Training episodes")
plt.ylabel("Success rate (%)")
plt.ylim(0, 100)
plt.grid(True, axis="y", alpha=0.3)
plt.show()
```

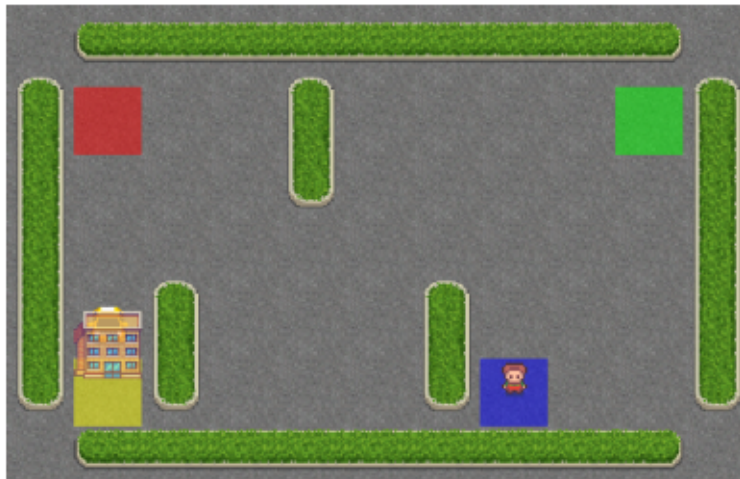
Visualising the environment

- `render_mode` is set to `rgb_array` instead of the continuous human window, since a notebook can't host a live render loop.
- One frame is captured and shown with `imshow`.

```
[9]: render_env = gym.make("Taxi-v4", render_mode="rgb_array")
render_env.reset(seed=0)
frame = render_env.render()
render_env.close()

plt.figure(figsize=(5, 5))
plt.imshow(frame)
plt.axis("off")
plt.title("Taxi-v4 environment")
plt.show()
```

Taxi-v4 environment



Visualising the agent's moves

- A single frame shows a state, not behaviour.
- The trained policy is rolled out greedily for one test episode, and every frame along the way is captured.
- At a 45 percent success rate, most seeds would time out at 200 steps, so a seed that finishes quickly is picked, purely so the filmstrip is readable.
- This choice of seed does not touch the Q-table or any reported metric.

```
[10]: def find_demo_seed(q_table, max_search=300, max_steps_display=15):
    for seed in range(max_search):
        search_env = gym.make("Taxi-v4")
        state, info = search_env.reset(seed=seed)
        done, terminated, steps = False, False, 0
        while not done and steps < max_steps_per_episode:
            action = np.argmax(q_table[state])
            state, reward, terminated, truncated, info = search_env.step(action)
            done = terminated or truncated
            steps += 1
        search_env.close()
        if terminated and steps <= max_steps_display:
            return seed
    return None
```

```
best_num_episodes = max(episode_configs)
best_q_table = results[best_num_episodes]["q_table"]
```

```

action_names = {0: "South", 1: "North", 2: "East", 3: "West", 4: "Pickup", 5: "Dropoff"}
demo_seed = find_demo_seed(best_q_table)

rollout_env = gym.make("Taxi-v4", render_mode="rgb_array")
state, info = rollout_env.reset(seed=demo_seed)

frames = [rollout_env.render()]
step_labels = ["Start"]
done = False

while not done:
    action = np.argmax(best_q_table[state])
    state, reward, terminated, truncated, info = rollout_env.step(action)
    done = terminated or truncated
    frames.append(rollout_env.render())
    step_labels.append(f"Step {len(step_labels)}: {action_names[action]}")

rollout_env.close()
print(f"Rollout used seed {demo_seed}, finished in {len(frames) - 1} steps,
      terminated = {terminated}")

cols = 5
rows = int(np.ceil(len(frames) / cols))
fig, axes = plt.subplots(rows, cols, figsize=(cols * 2.4, rows * 2.4))
axes = np.array(axes).reshape(-1)

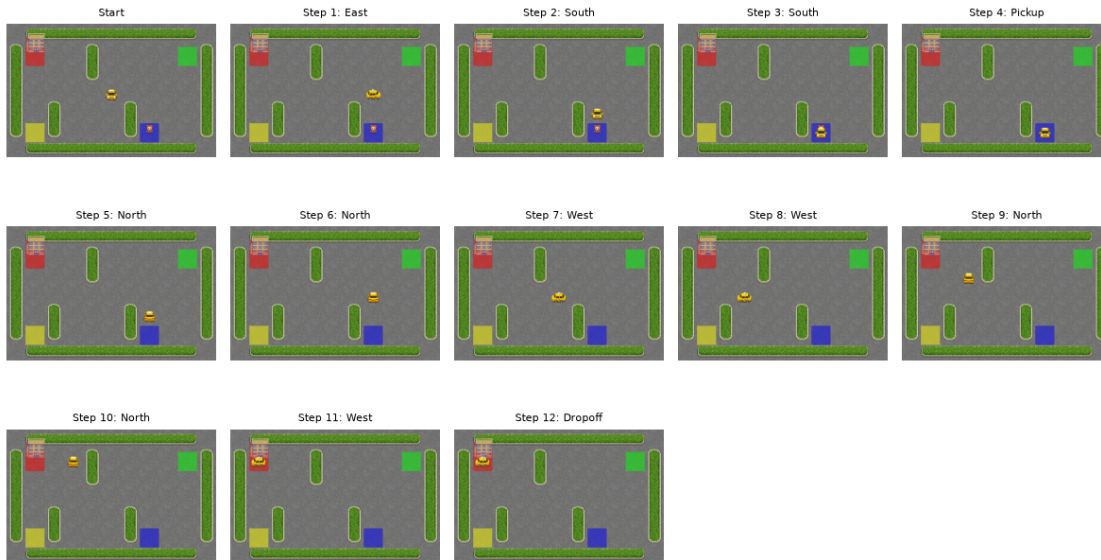
for i, ax in enumerate(axes):
    if i < len(frames):
        ax.imshow(frames[i])
        ax.set_title(step_labels[i], fontsize=8)
        ax.axis("off")

fig.suptitle("Greedy rollout of the learned policy (Taxi-v4)", y=1.02)
fig.tight_layout()
plt.show()

```

Rollout used seed 1, finished in 12 steps, terminated = True

Greedy rollout of the learned policy (Taxi-v4)



Extracting and visualising the learned policy

- The policy for a state is argmax over that state's row in the Q-table.
- The table used is the representative, seed-42, 10000-episode run, since it scored highest.
- Taxi's policy has two phases, drive to the passenger, then drive to the destination, shown here as two 5x5 grids.
- P and D are only correct at the passenger's own cell and the destination's own cell, elsewhere they mean the table has not converged for that state.

```
[11]: location_names = {0: "R", 1: "G", 2: "Y", 3: "B"}
      arrow_symbols = {0: "↓", 1: "↑", 2: "→", 3: "←", 4: "P", 5: "D"}

      taxi_env = gym.make("Taxi-v4").unwrapped
      pickup_location = 2    # Y
      dropoff_location = 0   # R
      passenger_in_taxi = 4

      print(f"Learned policy (from the {best_num_episodes}-episode run)\n")

      print(f"Phase 1: passenger waiting at {location_names[pickup_location]}, taxi_
            ↪should drive there and pick up")
      for row in range(5):
          line = ""
          for col in range(5):
              state = taxi_env.encode(row, col, pickup_location, dropoff_location)
```

```

        line += f" {arrow_symbols[best_q_table[state].argmax()]}"
    print(line)

print(f"\nPhase 2: passenger on board, taxi should drive to_
↔{location_names[dropoff_location]} and drop off")
for row in range(5):
    line = ""
    for col in range(5):
        state = taxi_env.encode(row, col, passenger_in_taxi, dropoff_location)
        line += f" {arrow_symbols[best_q_table[state].argmax()]}"
    print(line)

```

Learned policy (from the 10000-episode run)

Phase 1: passenger waiting at Y, taxi should drive there and pick up

```

← ↓ ← ↑ ←
↓ ↑ → ↑ ←
↓ ← ← → ↓
↓ ↑ ↓ ↓ ←
P → ← ← ↓

```

Phase 2: passenger on board, taxi should drive to R and drop off

```

D ← ↓ ↓ ←
↑ ↑ ↓ ↓ ←
→ ↑ ← ← ←
↑ ↑ ↑ ↑ ←
↑ ↑ ↑ ↑ ←

```

Inference

- Success rate, mean test reward, and the moving average all improve with more episodes, and none of them plateau early, this is a 500-state environment.
- Shorter runs stop training while epsilon is still high, so their episodes are still partly random when training ends, which explains their lower and noisier results.
- Cumulative reward keeps falling through training here, since per-step reward is mostly negative, so its slope, not its sign, is the signal worth reading.
- The moving average climbing from deeply negative toward zero is a cleaner progress signal than the cumulative total.
- Test success rate is a strict measure of convergence here, since Taxi has no randomness to blame a failure on.
- The policy grids get pickup and drop-off exactly right at the passenger's and destination's own cells, and most surrounding cells point the right way, the handful that don't are the least-visited states at this training budget.